

10

Алгоритмы параллельных вычислений

В этой главе

- Использование алгоритмов параллельных вычислений C++17.

В предыдущей главе мы рассмотрели усовершенствованное управление потоками и пулы потоков, а глава 8 была посвящена разработке конкурентного кода, где в качестве примеров использовались параллельные версии некоторых алгоритмов. В этой главе мы разберем алгоритмы параллельных вычислений (parallel algorithms), предоставляемые стандартом C++17.

10.1. Перевод стандартных библиотечных алгоритмов в режим параллельных вычислений

В стандарт C++17 к стандартной библиотеке C++ добавлена концепция *алгоритмов параллельных вычислений*. Она представлена в виде дополнительных переопределений многих функций, работающих с диапазонами, в числе которых `std::find`, `std::transform` и `std::reduce`. У параллельных версий такая же сигнатура, что

и у «обычных» однопоточных, за исключением добавления нового первого параметра, определяющего *политику выполнения*. Например:

```
std::vector<int> my_data;
std::sort(std::execution::par, my_data.begin(), my_data.end());
```

Политика выполнения `std::execution::par` указывает стандартной библиотеке, что данный вызов разрешено выполнять в качестве алгоритма параллельных вычислений с задействованием сразу нескольких потоков. Следует отметить, что это *разрешение*, а не *требование* — библиотека вольна по-прежнему выполнять код в одном потоке. Обратите внимание, что с заданием политики выполнения изменяются требования к сложности алгоритма, и они обычно слабее требований, предъявляемых к обычному алгоритму последовательных вычислений. Чтобы можно было воспользоваться преимуществами параллелизма счетеми, алгоритмы параллельных вычислений зачастую выполняют работу более общего характера. Если работу можно разделить и выполнить на 100 процессорах, то можно достичь чуть ли не 50-кратного ускорения, даже если общий объем работы в реализации удваивается.

Прежде чем перейти к самим алгоритмам, рассмотрим политики выполнения.

10.2. Политики выполнения

Стандартом предусмотрены три политики выполнения:

- ❑ `std::execution::sequenced_policy;`
- ❑ `std::execution::parallel_policy;`
- ❑ `std::execution::parallel_unsequenced_policy.`

Они представляют собой классы, определенные в заголовке `<execution>`. Там же определены три соответствующих объекта политик, которые передаются алгоритмам:

- ❑ `std::execution::seq;`
- ❑ `std::execution::par;`
- ❑ `std::execution::par_unseq.`

Нельзя рассчитывать на самостоятельное создание объектов из этих классов политик, поскольку из-за возможных особых требований по инициализации остается полагаться только на копирование этих трех объектов. В реализациях могут также определяться дополнительные политики выполнения с характерными для них особенностями поведения. Задавать свои собственные политики выполнения невозможно.

О влиянии этих политик на поведение алгоритмов речь пойдет в подразделе 10.2.1. В любой конкретной реализации также может предоставляться дополнительная политика выполнения с любой произвольной семантикой. А теперь рассмотрим последствия использования одной из стандартных политик выполнения, начиная с общих изменений для всех переопределений алгоритма, принимающего политику исключений.

10.2.1. Общие последствия от задания политики выполнения

Если передать политику выполнения одному из стандартных библиотечных алгоритмов, то она станет определять его поведение. Это повлияет на несколько характеристик поведения:

- ❑ сложность алгоритма;
- ❑ поведение при выдаче исключения;
- ❑ место, способ и время выполнения этапов работы алгоритма.

Влияние на сложность алгоритма

Если алгоритму предоставляется политика выполнения, его сложность может измениться: в дополнение к издержкам диспетчеризации выполнения параллельных вычислений, многими параллельными алгоритмами будет выполняться более весомый объем основных операций алгоритма (будь то *обмены, сравнения или использования предоставляемого функционального объекта*). Это предположительно улучшит производительность относительно затраченного времени.

Детали изменения сложности будут варьироваться для каждого алгоритма, но общая политика заключается в том, что если алгоритм определяет, что нечто произойдет за время, точно соответствующее вычислению какого-либо выражения (*some-expression*), или не превысит время вычисления какого-либо выражения, то переопределение алгоритма с применением политики выполнения ослабит это требование до $O(\text{some-expression})$. Это означает, что переопределение алгоритма с применением политики выполнения может выполнять некоторое кратное число операций, выполняемых его аналогом, не применяющим политику выполнения, где эта кратность будет зависеть от внутренних особенностей библиотеки и платформы, а не от данных, предоставленных алгоритму.

Поведение при выдаче исключения

Если алгоритм выполняется с заданной политикой и при этом выдается исключение, то последствия определяются этой политикой. При выдаче любых неперехваченных исключений всеми предоставляемыми стандартом политиками выполнения предусматривается вызов функции `std::terminate`. Единственным исключением, которое может быть выдано вызовом стандартного библиотечного алгоритма с одной из стандартных политик выполнения, является `std::bad_alloc`. Оно выдается, если библиотека не может получить достаточный объем ресурсов памяти для своих внутренних операций. Например, следующий вызов `std::for_each` без задания политики выполнения распространит исключение:

```
std::for_each(v.begin(),v.end(),[](auto x){ throw my_exception(); });
```

а соответствующий вызов с заданием политики выполнения завершит выполнение программы:

```
std::for_each(
    std::execution::seq,v.begin(),v.end(),
    [](auto x){ throw my_exception(); });
```

Это одно из главных различий между использованием `std::execution::seq` и не-предоставлением политики выполнения.

Где и когда выполняются этапы работы алгоритма

Это основной аспект политики выполнения и единственный аспект, определяющий различия между стандартными политиками выполнения. Политика определяет, какие агенты выполнения задействуются для реализации этапов алгоритма, будь то «обычные» потоки, векторные потоки, потоки графического процессора или что-либо еще. Политика выполнения также будет определять ограничения в порядке выполнения работы алгоритма: должны ли они запускаться в определенной последовательности, могут ли отдельные этапы работы алгоритма перемежаться или запускаться параллельно друг с другом и т. д.

Подробности каждой стандартной политики выполнения рассматриваются в подразделах 10.2.2–10.2.4. Сначала мы обсудим основную политику: `std::execution::sequenced_policy`.

10.2.2. `std::execution::sequenced_policy`

Последовательная политика (*sequenced policy*) не является политикой параллелизма: ее использование заставляет реализацию выполнять все операции в потоке, где была вызвана функция, то есть без распараллеливания. Но все же это политика выполнения, в силу чего она оказывает такое же влияние на алгоритмическую сложность и на выдачу исключений, как и все другие стандартные политики.

Все операции должны выполняться не только в одном потоке, но и в определенном порядке, то есть они не могут перемежаться. Конкретный порядок не устанавливается и может различаться для разных вызовов функции. В частности, не гарантируется, что порядок выполнения операций будет таким же, как и в соответствующем переопределении без задания политики выполнения. Например, следующий вызов `std::for_each` заполнит вектор числами 1–1000 в неопределенном порядке. Это будет отличаться от выполнения переопределения алгоритма, не предусматривающего задания политики выполнения, который сохранит числа в порядке возрастания:

```
std::vector<int> v(1000);
int count=0;
std::for_each(std::execution::seq, v.begin(), v.end(),
    [&](int& x){ x=++count; });
```

Числа могут быть сохранены и в порядке возрастания, но полагаться на это нельзя.

Это означает, что политика последовательного выполнения не предъявляет особых требований к используемым с алгоритмом итераторам, значениям и вызываемым объектам: они могут свободно использовать механизмы синхронизации и полагаться на все операции, вызываемые в том же потоке, но не могут рассчитывать на порядок выполнения этих операций.

10.2.3. `std::execution::parallel_policy`

Параллельная политика (`parallel policy`) предоставляет основное параллельное выполнение сразу в нескольких потоках. Операции можно выполнять либо в потоке, вызвавшем алгоритм, либо в потоках, созданных библиотекой. Операции, выполняемые в конкретном потоке, должны выполняться в определенном порядке и не перемежаться, но точный порядок не определен и от вызова к вызову может варьироваться. Конкретная операция будет выполняться в фиксированном потоке на протяжении всего времени.

Это накладывает дополнительные требования на итераторы, значения и вызываемые объекты, не используемые с алгоритмом при применении последовательной политики: они не должны приводить к состоянию гонки за данными при параллельном вызове и не должны полагаться на выполнение в том же потоке, что и любая другая операция, или же действительно полагаться на то, что не будут выполняться в том же потоке, что и любая другая операция.

Политикой параллельного выполнения можно воспользоваться в подавляющем большинстве случаев, когда не используется алгоритм стандартной библиотеки без применения политики выполнения. Обойдется без проблем только при наличии определенного порядка между необходимыми элементами или несинхронизированного обращения к совместно используемым данным. Увеличение на единицу всех значений, имеющихся в векторе, может быть выполнено в режиме параллельных вычислений:

```
std::for_each(std::execution::par, v.begin(), v.end(), [](auto& x){++x;});
```

А для заполнения вектора предыдущий пример с использованием политики параллельного выполнения не подойдет, поскольку возникнет неопределенное поведение:

```
std::for_each(std::execution::par, v.begin(), v.end(),
    [&](int& x){ x=++count; });
```

Здесь переменная `count` изменяется при каждом вызове лямбда-функции, поэтому, если бы библиотеке пришлось выполнять лямбда-функции сразу в нескольких потоках, возникло бы состояние гонки, что привело бы к неопределенному поведению. Это исключается требованиями, предъявляемыми к `std::execution::parallel_policy`: имеется в виду неопределенное поведение при выполнении предыдущего вызова, даже если библиотека не использует для этого вызова несколько потоков. Проявление или непроявление неопределенного поведения является статическим свойством вызова, не зависящим от особенностей реализации библиотеки. Но синхронизация между вызовами функции разрешена, поэтому вполне возможно придать поведению определенность, задав для `count` не обычный тип `int`, а `std::atomic<int>` или же воспользовавшись мьютексом. В таком случае использование политики параллельного выполнения будет бессмысленным, поскольку все вызовы станут последовательными, но в общем случае это позволит синхронизировать доступ к совместно используемому состоянию.

10.2.4. `std::execution::parallel_unsequenced_policy`

Политика параллельного неупорядоченного выполнения (parallel unsequenced policy) предоставляет библиотеке наибольший масштаб распараллеливания алгоритма в обмен на предъявление строжайших требований к итераторам, значениям и вызываемым объектам, используемым с алгоритмом.

Алгоритм, вызываемый с применением параллельной неупорядоченной политики, может выполнять этапы алгоритма в неопределенных потоках выполнения, неупорядоченных и непоследовательных относительно друг друга. Это означает, что операции теперь могут перемежаться в одном потоке, благодаря чему вторая операция будет запускаться в одном и том же потоке до завершения первой операции и может быть перенесена между потоками. То есть конкретная операция может начаться в одном потоке, продолжиться во втором, а завершиться в третьем.

Операции, предоставляемые алгоритму с использованием политики параллельного неупорядоченного выполнения, при их вызове итераторами, значениями и вызываемыми объектами, не должны использовать какие-либо формы синхронизации или вызовы любых функций, синхронизируемых с другими функциями, или же любых функций, с которыми синхронизируется другой код.

Это означает, что операции должны работать только с соответствующим элементом или с любыми данными, доступными на основе этого элемента, и не должны изменять какое-либо состояние, совместно используемое потоками или элементами.

Подробности будут раскрыты на нескольких примерах чуть позже. А теперь посмотрим на сами параллельные алгоритмы.

10.3. Параллельные алгоритмы из стандартной библиотеки C++

Большинство алгоритмов из заголовков `<algorithm>` и `<numeric>` имеют переопределения, воспринимающие политику выполнения. В их числе: `all_of`, `any_of`, `none_of`, `for_each`, `for_each_n`, `find`, `find_if`, `find_end`, `find_first_of`, `adjacent_find`, `count`, `count_if`, `mismatch`, `equal`, `search`, `search_n`, `copy`, `copy_n`, `copy_if`, `move`, `swap_ranges`, `transform`, `replace`, `replace_if`, `replace_copy`, `replace_copy_if`, `fill`, `fill_n`, `generate`, `generate_n`, `remove`, `remove_if`, `remove_copy`, `remove_copy_if`, `unique`, `unique_copy`, `reverse`, `reverse_copy`, `rotate`, `rotate_copy`, `is_partitioned`, `partition`, `stable_partition`, `partition_copy`, `sort`, `stable_sort`, `partial_sort`, `partial_sort_copy`, `is_sorted`, `is_sorted_until`, `nth_element`, `merge`, `inplace_merge`, `includes`, `set_union`, `set_intersection`, `set_difference`, `set_symmetric_difference`, `is_heap`, `is_heap_until`, `min_element`, `max_element`, `minmax_element`, `lexicographical_compare`, `reduce`, `transform_reduce`, `exclusive_scan`, `inclusive_scan`, `transform_exclusive_scan`, `transform_inclusive_scan` и `adjacent_difference`.

В этом обширном списке перечислены почти все алгоритмы стандартной библиотеки C++, которые могут быть распараллелены. Исключением является такой алгоритм, как `std::accumulate`, представляющий собой сугубо последовательное накопление. Его обобщенный контрипартнер `std::reduce` все же фигурирует в списке,

по с соответствующей оговоркой в стандарте: операция сокращения не является ни ассоциативной, ни коммутативной, результат из-за неопределенного порядка выполнения операций может быть недетерминированным.

Для каждого указанного в списке алгоритма каждое «обычное» переопределение обладает новым вариантом, принимающим в качестве первого аргумента политику выполнения. Затем за этой политикой выполнения следуют соответствующие аргументы для «обычного» алгоритма. Например, у `std::sort` есть два «обычных» переопределения без задания политики выполнения:

```
template<class RandomAccessIterator>
void sort(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
void sort(
    RandomAccessIterator first, RandomAccessIterator last, Compare comp);
```

Соответственно, у него есть и два переопределения с заданием политики выполнения:

```
template<class ExecutionPolicy, class RandomAccessIterator>
void sort(
    ExecutionPolicy&& exec,
    RandomAccessIterator first, RandomAccessIterator last);

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void sort(
    ExecutionPolicy&& exec,
    RandomAccessIterator first, RandomAccessIterator last, Compare comp);
```

Сигнатуры, используемые в вариантах с заданием политики выполнения и без ее задания, имеют одно весьма важное различие, оказывающее влияние лишь на некоторые алгоритмы: если в «обычном» алгоритме допускаются *итераторы ввода* (*input iterators*) или *итераторы вывода* (*output iterators*), то в переопределениях с заданием политики выполнения вместо них требуются *однонаправленные итераторы* (*forward iterators*). Дело в том, что входные итераторы в основном являются односторонними: доступ можно получить только к текущему элементу и нельзя сохранить итераторы для предыдущих элементов. Аналогично выходные итераторы позволяют вести запись только текущего элемента: их нельзя передвинуть на запись более позднего элемента, а затем вернуться и записать предыдущий элемент.

Категории итераторов в стандартной библиотеке C++

В стандартной библиотеке C++ определено пять категорий итераторов: итераторы ввода (*input iterators*), итераторы вывода (*output iterators*), однонаправленные итераторы (*forward iterators*), двунаправленные итераторы (*bidirectional iterators*) и итераторы произвольного доступа (*random access iterators*).

Итераторы ввода являются односторонними итераторами для извлечения значений. Обычно они используются для ввода с консоли, или из сети, или из генерированных

последовательностей. Продвижение итератора ввода делает любые его копии недействительными.

Итераторы вывода являются однопроходными итераторами для записи значений. Обычно они используются для вывода данных в файлы или для добавления значений в контейнер. Продвижение итератора вывода делает любые его копии недействительными.

Однонаправленные итераторы являются многопроходными для однонаправленного последовательного обхода существующих данных. Следовательно, итератор нельзя вернуть на предыдущий элемент, можно сохранить копии и воспользоваться ими для ссылок на предыдущие элементы. Однонаправленные итераторы возвращают ссылки на элементы, следовательно, ими можно воспользоваться как для чтения, так и для записи (если целевой элемент не относится к типу `const`).

Двухнаправленные итераторы, как и однонаправленные, являются многопроходными, но их также можно заставить повернуться вспять для доступа к предыдущим элементам.

Итераторы произвольного доступа являются многопроходными, способными к переходам вперед-назад, как и двухнаправленные итераторы, но переходы вперед-назад осуществляются шагами шире одного элемента. Можно напрямую получить доступ к элементам со смещением, используя индексный оператор массива.

Итак, исходя из «обычной» сигнатуры для `std::copy`:

```
template<class InputIterator, class OutputIterator>
OutputIterator copy(
    InputIterator first, InputIterator last, OutputIterator result);
```

получаем переопределение с заданием политики выполнения:

```
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 copy(
    ExecutionPolicy&& policy,
    ForwardIterator1 first, ForwardIterator1 last,
    ForwardIterator2 result);
```

Хотя с точки зрения компилятора наименование параметров шаблона не влечет за собой никаких прямых последствий, такие последствия имеются с точки зрения стандартов C++: имена параметров шаблона для алгоритмов стандартной библиотеки обозначают семантические ограничения на типы и алгоритмы с указанной семантикой будут зависеть от операций, подразумеваемых имеющимися ограничениями. Если сравнить итератор ввода и однонаправленный итератор, то первый позволяет разыменовывать итератор для возврата прокси-типа, который можно преобразовать в тип значения итератора, а второй требует, чтобы при разыменовании итератора возвращалась реальная ссылка на значение и все одинаковые итераторы возвращали ссылку на одно и то же значение.

Для параллельных вычислений это играет важную роль: это значит, что итераторы можно свободно и повсеместно копировать и использовать эквивалентным

образом. Кроме того, важно и требование о том, что приращение однонаправленного итератора не делает недействительными другие копии, поскольку это означает, что отдельно взятые потоки могут работать со своими собственными копиями итераторов, задавая им приращение при необходимости, не беспокоясь об аннулировании итераторов других потоков. Если переопределение с помощью политики выполнения позволяет не использовать итераторы ввода, это заставит любые потоки применять последовательную форму доступа к единственному итератору, который был задействован для считывания исходной последовательности. Это, конечно же, приведет к ограничению возможностей по распараллеливанию.

Рассмотрим несколько конкретных примеров.

10.3.1. Примеры использования параллельных алгоритмов

Самый простой пример — параллельный цикл: выполнение каких-либо действий над каждым элементом контейнера. Это классический пример сугубо параллельного сценария: каждый элемент независим от всех остальных, следовательно, предоставляется максимальная возможность для распараллеливания. При использовании компилятора, поддерживающего OpenMP, можно воспользоваться следующим кодом:

```
#pragma omp parallel for
for(unsigned i=0;i<v.size();++i){
    do_stuff(v[i]);
}
```

А не пользуя алгоритмы стандартной библиотеки C++, вместо него можно написать код:

```
std::for_each(std::execution::par,v.begin(),v.end(),do_stuff);
```

Он приведет к разбиению элементов на диапазоны между внутренними потоками, созданными библиотекой, и к вызову функции `do_stuff(x)` в отношении каждого элемента `x`, входящего в диапазон. Способ разбиения этих элементов по потокам — особенность реализации.

Выбор политики выполнения

Наиболее часто приходится иметь дело с политикой `std::execution::par`, если только в вашей реализации не предоставляется нестандартная политика, более полно отвечающая вашим потребностям. Если ваш код подходит для распараллеливания, то он должен работать с `std::execution::par`. В некоторых случаях вы можете не использовать `std::execution::par_unseq`. Это может вообще ни к чему не привести (ни одна из стандартных политик выполнения не дает гарантий по достигаемому уровню распараллеливания), но может дать библиотеке дополнительные возможности повышения производительности кода за счет изменения порядка выполнения и чередования задач взамен наложения на код более жестких требований.

Наиболее заметным из таких требований является отсутствие синхронизации при обращении к элементам плн при выполнении операций над элементами. Это означает невозможность применения мьютексов, плн атомарных переменных, плн любого другого механизма, рассмотренного в предыдущих главах, который обеспечивает безопасность обращения из нескольких потоков. Вместо этого нужно полагаться на то, что сам алгоритм не станет обращаться к одному и тому же элементу сразу из нескольких потоков и использовать внешнюю синхронизацию за пределами вызова параллельного алгоритма для предотвращения обращения к данным со стороны других потоков.

В листинге 10.1 показан код, которым можно воспользоваться с политикой выполнения `std::execution::par`, но не с `std::execution::par_unseq`. Использование для синхронизации внутреннего мьютекса означает, что попытка воспользоваться политикой `std::execution::par_unseq` приведет к неопределенному поведению.

Листинг 10.1. Применение параллельных алгоритмов к классу с внутренней синхронизацией

```
class X{
    mutable std::mutex m;
    int data;
public:
    X():data(0){}
    int get_value() const{
        std::lock_guard guard(m);
        return data;
    }
    void increment(){
        std::lock_guard guard(m);
        ++data;
    }
};

void increment_all(std::vector<X>& v){
    std::for_each(std::execution::par, v.begin(), v.end(),
        [](X& x){
            x.increment();
        });
}
```

В листинге 10.2 показан альтернативный вариант, которым можно воспользоваться с политикой выполнения `std::execution::par_unseq`. В этом случае вместо внутреннего поэлементного мьютекса используется мьютекс, действие которого распространяется на весь контейнер.

Листинг 10.2. Применение параллельных алгоритмов к классу без внутренней синхронизации

```
class Y{
    int data;
public:
    Y():data(0){}
    int get_value() const{
        return data;
    }
}
```

```

    }
    void increment(){
        ++data;
    }
};
class ProtectedY{
    std::mutex m;
    std::vector<Y> v;
public:
    void lock(){
        m.lock();
    }
    void unlock(){
        m.unlock();
    }
    std::vector<Y>& get_vec(){
        return v;
    }
};
void increment_all(ProtectedY& data){
    std::lock_guard guard(data);
    auto& v=data.get_vec();
    std::for_each(std::execution::par_unseq,v.begin(),v.end(),
        [](Y& y){
            y.increment();
        });
}

```

Теперь в листинге 10.2 обращение к элементу происходит без синхронизации, и можно без каких-либо опасений воспользоваться политикой `std::execution::par_unseq`. Недостаток такого решения проявляется в том, что при конкурентном доступе из других потоков, не входящих в сферу вызова параллельного алгоритма, приходится ждать завершения всей операции, в отличие от поэлементной детализации, имеющейся в коде листинга 10.1.

А теперь рассмотрим более реалистичный пример, показывающий порядок использования параллельных алгоритмов: подсчет количества посещений сайта.

10.3.2. Подсчет посещений

Предположим, что у нас есть запущенный деловой сайт, в журнале которого содержатся миллионы записей. Нужно обработать регистрационные записи, чтобы увидеть сводные данные: количество обращений каждой страницы, откуда они были, какие браузеры использовались для обращения к сайту и т. д. Анализ регистрационных записей состоит из двух частей: обработки каждой строки для извлечения нужной информации и получения сводных данных. Это идеальный сценарий для использования параллельных алгоритмов, поскольку обработка каждой отдельно взятой строки проводится независимо от чего бы то ни было, а создание сводки, если конечные итоги верны, может вестись по частям.

Получается та задача, для которой и разработана функция `transform_reduce`. Как ее можно использовать для решения данной задачи, показано в листинге 10.3.

Листинг 10.3. Использование `transform_reduce` для подсчета посещений страниц на сайте

```
#include <vector>
#include <string>
#include <unordered_map>
#include <numeric>
struct log_info {
    std::string page;
    time_t visit_time;
    std::string browser;
    // любые другие поля
};

extern log_info parse_log_line(std::string const &line);    ←❶

using visit_map_type= std::unordered_map<std::string, unsigned long long>;

visit_map_type
count_visits_per_page(std::vector<std::string> const &log_lines) {

    struct combine_visits {
        visit_map_type
        operator()(visit_map_type lhs, visit_map_type rhs) const {    ←❷
            if(lhs.size() < rhs.size())
                std::swap(lhs, rhs);
            for(auto const &entry : rhs) {
                lhs[entry.first] += entry.second;
            }
            return lhs;
        }

❸→ visit_map_type operator()(log_info log, visit_map_type map) const{
        ++map[log.page];
        return map;
    }
❹→ visit_map_type operator()(visit_map_type map, log_info log) const{
        ++map[log.page];
        return map;
    }
❺→ visit_map_type operator()(log_info log1, log_info log2) const{
        visit_map_type map;
        ++map[log1.page];
        ++map[log2.page];
        return map;
    }
};

return std::transform_reduce(    ←❺
    std::execution::par, log_lines.begin(), log_lines.end(),
    visit_map_type(), combine_visits(), parse_log_line);
}
```

Предположим, что для извлечения нужной информации из регистрационной записи вы располагаете некоей функцией `parse_log_line` ❶, ваша функция `count_visits_per_page` служит простой оболочкой, в которую заключен вызов `std::transform_reduce` ❷. Сложности исходят из операции *предварительной обработки*: нужно скомбинировать две структуры `log_info` для создания отображения, структуру `log_info` и отображение (тем или иным образом) и два отображения. Следовательно, это означает, что ваш функциональный объект `combine_visits` нуждается для оператора функционального вызова в четырех переопределениях, ❸, ❹, ❺ и ❻, что не позволяет обойтись простой лямбда-функцией, несмотря на то что реализация этих четырех переопределений довольно проста.

Поэтому для выполнения данных вычислений в параллельном режиме (поскольку была передана политика `std::execution::par`) реализацией `std::transform_reduce` будет непользоваться доступное оборудование. Как уже было показано в предыдущей главе, создание своего собственного подобного алгоритма — задача непростая, следовательно, вам предоставляется возможность переложить всю трудную работу по реализации параллелизма на разработчиков стандартной библиотеки и сосредоточиться на требуемом результате.

Резюме

В данной главе мы рассмотрели параллельные алгоритмы, доступные в стандартной библиотеке C++, и порядок их использования. Уделили внимание различным политикам выполнения, влиянию выбора политики выполнения на поведение алгоритма и на ограничения для вашего кода. Затем разобрали примеры возможного применения этих алгоритмов в реальном коде.